

LABORATORY WORK #2

Normalization and DDL

In this lab I worked with database normalization and DDL commands. I first normalized the booking receipt table and then created the airport database tables and relationships.

PART I. NORMALIZATION

1. What normal form is violated?

The original table is:

```
Booking_receipt(booking_id, passenger_full_name, passenger_passport_number,
flight_number, departure_airport_name, departure_city, arrival_airport_name,
arrival_city, airline_name, seat_numbers, ticket_price)
```

The main problem is with 1NF. The column `seat_numbers` can contain several values in one cell, for example "12A, 12B". In 1NF one cell should contain only one value. Because of this, the table is not in 1NF.

There is also repeated passenger information when one booking has several passengers. After fixing 1NF, this problem will be handled when we move to 2NF and 3NF.

Problem	Column	Why it is a problem
1NF	seat_numbers	Several seat values can be stored in one cell.
Repeated data	passenger_full_name, passenger_passport_number	The same passenger data can be repeated for one booking.
Later normalization	flight_number, airport data, airline_name	These attributes describe the flight, airport or airline and should be stored separately.

2. Normalization step by step

Step 1 - 1NF

To get 1NF, I changed `seat_numbers` into a single `seat_number` value. If a booking has two seats, I use two rows instead of putting both seats into one cell.

booking_id	passport_number	passenger_name	flight_number	departure_airport	departure_city	arrival_airport	arrival_city	airline	seat_number	ticket_price
B001	P1001	Ayan Karimov	KC101	Almaty International Airport	Almaty	Astana International Airport	Astana	Kazakhstan Airlines	12A	45000.00
B001	P1002	Dana Karimova	KC101	Almaty International Airport	Almaty	Astana International Airport	Astana	Kazakhstan Airlines	12B	45000.00
B002	P1003	Madi	DV210	Almaty International	Almaty	Aktau	Aktau	FlySte	8C	38000.

		Sarsenov		l Airport		Airport		p		00
B003	P1004	Aliya Nur	DV210	Almaty International Airport	Almaty	Aktau Airport	Aktau	FlyStep	8D	38000.00
B004	P1005	Timur Bek	FS305	Astana International Airport	Astana	Shymkent Airport	Shymkent	SkyNomad	4A	32000.00

Primary key idea for this example: (booking_id, passport_number). Every seat value is now atomic.

Step 2 - 2NF

For 2NF I remove partial dependencies. Passenger name depends on passport_number. Flight number and price depend on booking_id. So I separate passenger data and booking data.

Table	Attributes
Passenger	<u>passport_number</u> , passenger_name
Booking	<u>booking_id</u> , flight_number, ticket_price
BookingPassenger	<u>booking_id</u> , <u>passport_number</u> , seat_number

Here BookingPassenger connects a booking with a passenger. The two key columns are foreign keys to Booking and Passenger.

Step 3 - 3NF

For 3NF I remove transitive dependencies. Flight number determines the airport route and airline. Airport id determines airport information. Airline id determines airline information. Because of this, I make separate Flight, Airport and Airline tables.

3NF table	Attributes / keys
Booking	<u>booking_id</u> , flight_number (FK), ticket_price
Passenger	<u>passport_number</u> , passenger_name
BookingPassenger	<u>booking_id</u> (FK), <u>passport_number</u> (FK), seat_number
Flight	<u>flight_number</u> , departure_airport_id (FK), arrival_airport_id (FK), airline_id (FK)
Airport	<u>airport_id</u> , airport_name, city
Airline	<u>airline_id</u> , airline_name

After this split, the same airport or airline information does not have to be written in every booking row. This is the main reason for using 3NF.

3. Functional dependencies

Step	Functional dependency	Meaning
1NF	(booking_id, passport_number) -> seat_number	A seat is connected to one passenger inside one booking.
2NF	booking_id -> flight_number,	The booking determines its flight and

	ticket_price	price.
2NF	passport_number -> passenger_name	Passport number identifies the passenger.
3NF	flight_number -> departure_airport_id, arrival_airport_id, airline_id	A flight determines its route and airline.
3NF	airport_id -> airport_name, city	Airport id identifies airport information.
3NF	airline_id -> airline_name	Airline id identifies the airline.

4. Is Flights in 3NF?

In my airport database the primary key of Flights is flight_id.

```
flight_id -> sch_departure_time, sch_arrival_time, departing_airport_id,
arriving_airport_id, departing_gate, arriving_gate, airline_id,
act_departure_time, act_arrival_time, created_at, updated_at
```

Flights is in 3NF. The columns departing_airport_id, arriving_airport_id and airline_id are foreign keys. They are used to get data from Airport and Airline tables. Flights does not also store airport_name, city or airline_name, so there is no extra transitive dependency between non-key columns inside Flights.

Therefore I consider Flights to be in 3NF.

The screenshot shows a PostgreSQL IDE interface. The top bar indicates the database is 'phonebook_db/postgres@PostgreSQL 18'. The main query editor contains the following SQL code:

```

31     departing_airport_id INT NOT NULL REFERENCES Airport(airport_id),
32     arriving_airport_id INT NOT NULL REFERENCES Airport(airport_id),
33     departing_gate TEXT NOT NULL,
34     arriving_gate VARCHAR(50) NOT NULL,
35     airline_id INT NOT NULL REFERENCES Airline(airline_id),
36     act_departure_time TIMESTAMP NOT NULL,
37     act_arrival_time TIMESTAMP NOT NULL,
38     created_at TIMESTAMP NOT NULL,
39     updated_at TIMESTAMP NOT NULL
40 );
41
42 -- 4. Проверка структуры таблицы
43 SELECT
44     column_name,
45     data_type,
46     is_nullable
47 FROM information_schema.columns
48 WHERE table_name = 'flights';

```

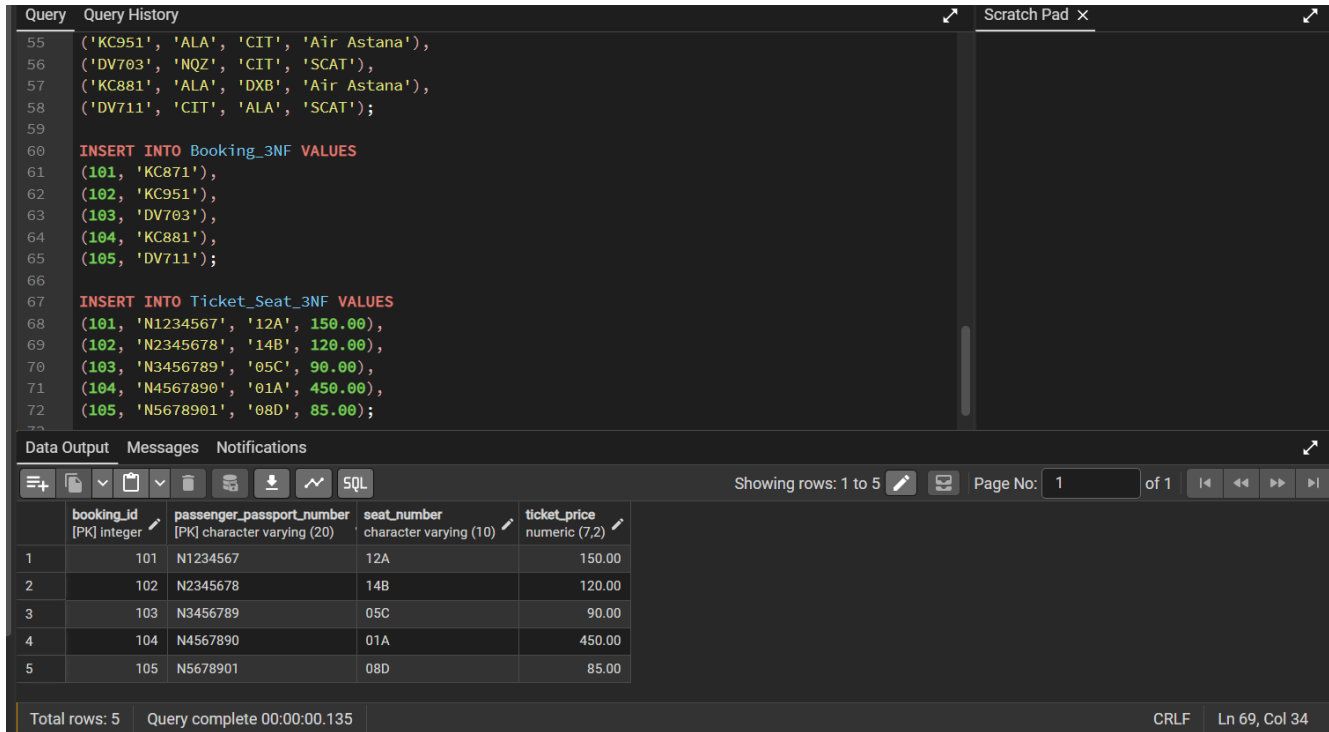
The bottom panel shows the 'Data Output' tab with the results of the query. It displays a table with 3 columns: column_name, data_type, and is_nullable. The results are as follows:

	column_name	data_type	is_nullable
1	flight_id	integer	NO
2	sch_departure_time	timestamp without time zone	NO
3	sch_arrival_time	timestamp without time zone	NO
4	departing_airport_id	integer	NO
5	arriving_airport_id	integer	NO
6	created_at	timestamp without time zone	NO

The status bar at the bottom indicates 'Total rows: 12', 'Query complete 00:00:00.101', and 'Ln 48, Col 30'.

Task 5: Implementing 3NF Tables and Inserting Data

In this task, I implemented the 3NF tables designed in Task 2 using SQL CREATE TABLE statements.



The screenshot shows a SQL IDE interface. The top pane displays a query with two INSERT statements. The first statement inserts data into the 'Booking_3NF' table, and the second statement inserts data into the 'Ticket_Seat_3NF' table. The bottom pane shows the 'Data Output' window, which displays the results of the query in a table format.

```
Query History
55 ('KC951', 'ALA', 'CIT', 'Air Astana'),
56 ('DV703', 'NQZ', 'CIT', 'SCAT'),
57 ('KC881', 'ALA', 'DXB', 'Air Astana'),
58 ('DV711', 'CIT', 'ALA', 'SCAT');
59
60 INSERT INTO Booking_3NF VALUES
61 (101, 'KC871'),
62 (102, 'KC951'),
63 (103, 'DV703'),
64 (104, 'KC881'),
65 (105, 'DV711');
66
67 INSERT INTO Ticket_Seat_3NF VALUES
68 (101, 'N1234567', '12A', 150.00),
69 (102, 'N2345678', '14B', 120.00),
70 (103, 'N3456789', '05C', 90.00),
71 (104, 'N4567890', '01A', 450.00),
72 (105, 'N5678901', '08D', 85.00);
73
```

Data Output Messages Notifications

Showing rows: 1 to 5 Page No: 1 of 1

	booking_id [PK] integer	passenger_passport_number [PK] character varying (20)	seat_number character varying (10)	ticket_price numeric (7,2)
1	101	N1234567	12A	150.00
2	102	N2345678	14B	120.00
3	103	N3456789	05C	90.00
4	104	N4567890	01A	450.00
5	105	N5678901	08D	85.00

Total rows: 5 Query complete 00:00:00.135 CRLF Ln 69, Col 34

PART II. DDL

1-4. CREATE TABLE, PRIMARY KEY and NOT NULL

Below is the SQL that I used. First I create all requested tables. Every attribute from the task is NOT NULL and every table has a primary key.

```
CREATE TABLE airline_info (  
    airline_id INT NOT NULL PRIMARY KEY,  
    airline_code VARCHAR(30) NOT NULL,  
    airline_name VARCHAR(50) NOT NULL,  
    airline_country VARCHAR(50) NOT NULL,  
    created_at TIMESTAMP NOT NULL,  
    updated_at TIMESTAMP NOT NULL,  
    info VARCHAR(50) NOT NULL  
);  
  
CREATE TABLE airport (  
    airport_id INT NOT NULL PRIMARY KEY,  
    airport_name VARCHAR(50) NOT NULL,  
    country VARCHAR(50) NOT NULL,  
    state VARCHAR(50) NOT NULL,  
    city VARCHAR(50) NOT NULL,  
    created_at TIMESTAMP NOT NULL,
```

```

        updated_at TIMESTAMP NOT NULL
    );

CREATE TABLE passengers (
    passenger_id INT NOT NULL PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    date_of_birth DATE NOT NULL,
    gender VARCHAR(50) NOT NULL,
    country_of_citizenship VARCHAR(50) NOT NULL,
    country_of_residence VARCHAR(50) NOT NULL,
    passport_number VARCHAR(20) NOT NULL,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL
);

CREATE TABLE flights (
    flight_id INT NOT NULL PRIMARY KEY,
    sch_departure_time TIMESTAMP NOT NULL,
    sch_arrival_time TIMESTAMP NOT NULL,
    departing_airport_id INT NOT NULL,
    arriving_airport_id INT NOT NULL,
    departing_gate VARCHAR(50) NOT NULL,
    arriving_gate VARCHAR(50) NOT NULL,
    airline_id INT NOT NULL,
    act_departure_time TIMESTAMP NOT NULL,
    act_arrival_time TIMESTAMP NOT NULL,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL
);

CREATE TABLE booking (
    booking_id INT NOT NULL PRIMARY KEY,
    flight_id INT NOT NULL,
    passenger_id INT NOT NULL,
    booking_platform VARCHAR(50) NOT NULL,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL,
    status VARCHAR(50) NOT NULL,
    price DECIMAL(7,2) NOT NULL
);

CREATE TABLE booking_flight (
    booking_flight_id INT NOT NULL PRIMARY KEY,
    booking_id INT NOT NULL,
    flight_id INT NOT NULL,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL
);

CREATE TABLE baggage_check (
    baggage_check_id INT NOT NULL PRIMARY KEY,
    check_result VARCHAR(50) NOT NULL,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL,
    booking_id INT NOT NULL,
    passenger_id INT NOT NULL
);

```

```

CREATE TABLE baggage (
    baggage_id INT NOT NULL PRIMARY KEY,
    weight_in_kg DECIMAL(4,2) NOT NULL,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL,
    booking_id INT NOT NULL
);

CREATE TABLE boarding_pass (
    boarding_pass_id INT NOT NULL PRIMARY KEY,
    booking_id INT NOT NULL,
    seat VARCHAR(50) NOT NULL,
    boarding_time TIMESTAMP NOT NULL,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL
);

CREATE TABLE security_check (
    security_check_id INT NOT NULL PRIMARY KEY,
    check_result VARCHAR(20) NOT NULL,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL,
    passenger_id INT NOT NULL
);

```

The screenshot shows a database management tool interface with a dark theme. The main window is divided into two panes. The left pane, titled 'Query', contains a list of SQL queries with line numbers 92 through 109. The right pane, titled 'Scratch Pad', is empty. Below the query panes is a toolbar with icons for various database operations. At the bottom, there is a 'Data Output' section showing a table with 6 rows and 1 column. The table has a header row with 'table_name' and a lock icon. The rows contain the following values: 'airline_info', 'airport', 'airport_3nf', 'baggage', 'baggage_che...', and 'boarding_pass'.

Query History

```

92     booking_id INT NOT NULL,
93     seat VARCHAR(50) NOT NULL,
94     boarding_time TIMESTAMP NOT NULL,
95     created_at TIMESTAMP NOT NULL,
96     updated_at TIMESTAMP NOT NULL
97 );
98 CREATE TABLE security_check (
99     security_check_id INT NOT NULL PRIMARY KEY,
100    check_result VARCHAR(20) NOT NULL,
101    created_at TIMESTAMP NOT NULL,
102    updated_at TIMESTAMP NOT NULL,
103    passenger_id INT NOT NULL
104 );
105
106 SELECT table_name
107 FROM information_schema.tables
108 WHERE table_schema = 'public'
109 ORDER BY table_name;

```

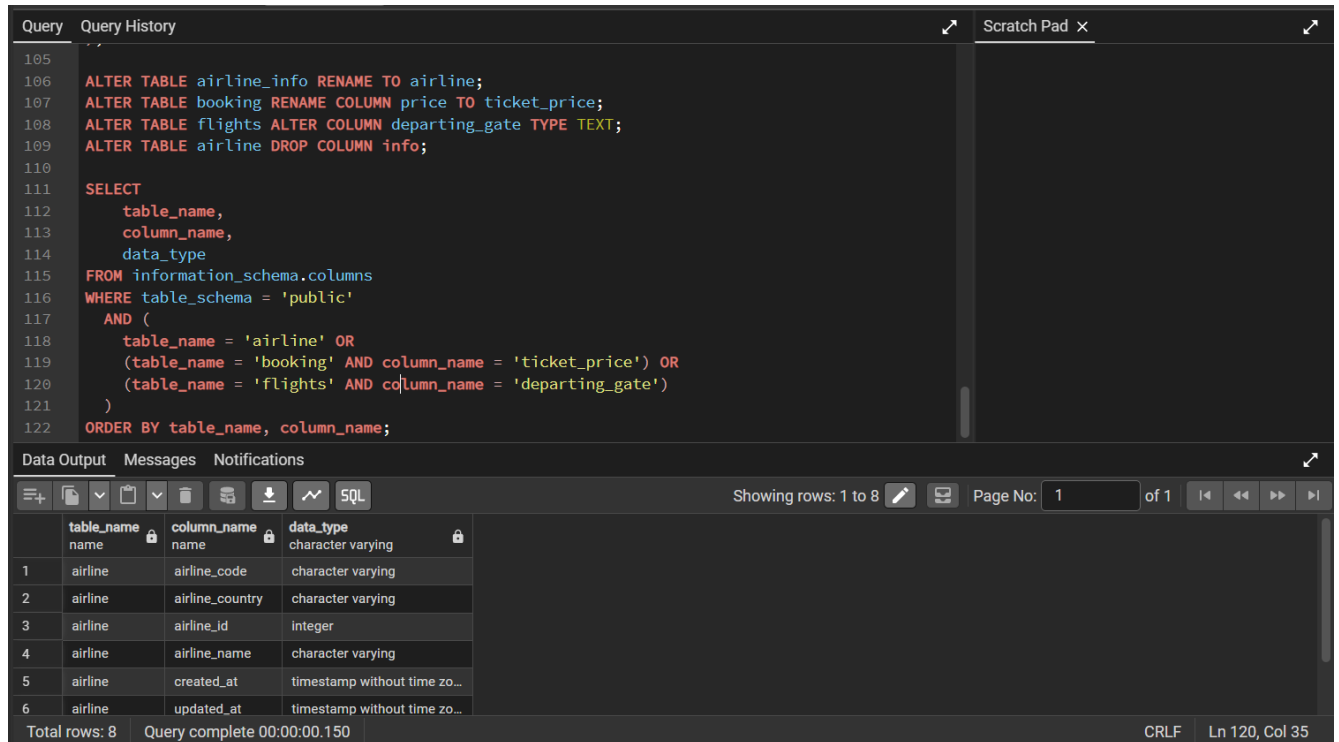
Data Output Messages Notifications

Showing rows: 1 to 15 Page No: 1 of 1

table_name
1 airline_info
2 airport
3 airport_3nf
4 baggage
5 baggage_che...
6 boarding_pass

5-8. Required ALTER TABLE commands

```
ALTER TABLE airline_info RENAME TO airline;
ALTER TABLE booking RENAME COLUMN price TO ticket_price;
ALTER TABLE flights ALTER COLUMN departing_gate TYPE TEXT;
ALTER TABLE airline DROP COLUMN info;
```



The screenshot shows a SQL IDE interface. The top pane displays a query with line numbers 105 to 122. The query includes several ALTER TABLE statements and a SELECT statement. The bottom pane shows the results of the query in a table format.

```
105
106 ALTER TABLE airline_info RENAME TO airline;
107 ALTER TABLE booking RENAME COLUMN price TO ticket_price;
108 ALTER TABLE flights ALTER COLUMN departing_gate TYPE TEXT;
109 ALTER TABLE airline DROP COLUMN info;
110
111 SELECT
112     table_name,
113     column_name,
114     data_type
115 FROM information_schema.columns
116 WHERE table_schema = 'public'
117     AND (
118         table_name = 'airline' OR
119         (table_name = 'booking' AND column_name = 'ticket_price') OR
120         (table_name = 'flights' AND column_name = 'departing_gate')
121     )
122 ORDER BY table_name, column_name;
```

	table_name	column_name	data_type
1	airline	airline_code	character varying
2	airline	airline_country	character varying
3	airline	airline_id	integer
4	airline	airline_name	character varying
5	airline	created_at	timestamp without time zo...
6	airline	updated_at	timestamp without time zo...

Total rows: 8 Query complete 00:00:00.150 CRLF Ln 120, Col 35

9. Foreign keys / relationships

I added the relationships after creating the tables. Airport is connected to Flights two times, because a flight has both a departure airport and an arrival airport.

```
ALTER TABLE security_check
ADD CONSTRAINT fk_security_check_passenger
FOREIGN KEY (passenger_id) REFERENCES passengers (passenger_id);

ALTER TABLE booking
ADD CONSTRAINT fk_booking_passenger
FOREIGN KEY (passenger_id) REFERENCES passengers (passenger_id);

ALTER TABLE baggage_check
ADD CONSTRAINT fk_baggage_check_passenger
FOREIGN KEY (passenger_id) REFERENCES passengers (passenger_id);

ALTER TABLE baggage_check
ADD CONSTRAINT fk_baggage_check_booking
FOREIGN KEY (booking_id) REFERENCES booking (booking_id);

ALTER TABLE boarding_pass
ADD CONSTRAINT fk_boarding_pass_booking
FOREIGN KEY (booking_id) REFERENCES booking (booking_id);
```

```

ALTER TABLE booking_flight
ADD CONSTRAINT fk_booking_flight_booking
FOREIGN KEY (booking_id) REFERENCES booking(booking_id);

ALTER TABLE booking_flight
ADD CONSTRAINT fk_booking_flight_flight
FOREIGN KEY (flight_id) REFERENCES flights(flight_id);

ALTER TABLE booking
ADD CONSTRAINT fk_booking_flight
FOREIGN KEY (flight_id) REFERENCES flights(flight_id);

ALTER TABLE flights
ADD CONSTRAINT fk_flights_departing_airport
FOREIGN KEY (departing_airport_id) REFERENCES airport(airport_id);

ALTER TABLE flights
ADD CONSTRAINT fk_flights_arriving_airport
FOREIGN KEY (arriving_airport_id) REFERENCES airport(airport_id);

ALTER TABLE flights
ADD CONSTRAINT fk_flights_airline
FOREIGN KEY (airline_id) REFERENCES airline(airline_id);

```

The screenshot shows a PostgreSQL IDE interface. The top panel displays a SQL query (lines 148-167) that checks for foreign key constraints. The query uses the `information\_schema` tables to find all foreign key constraints in the `public` schema. The bottom panel shows the results of the query in a table format.

Query:

```

148 ADD CONSTRAINT fk_flights_arriving_airport FOREIGN KEY (arriving_airport_id) REFERENCES Ai
149 ADD CONSTRAINT fk_flights_airline FOREIGN KEY (airline_id) REFERENCES Airline(airline_id);
150
151 -- 5. Запрос для проверки созданных связей
152 SELECT
153     tc.table_name,
154     tc.constraint_name,
155     kcu.column_name,
156     ccu.table_name AS referenced_table,
157     ccu.column_name AS referenced_column
158 FROM information_schema.table_constraints AS tc
159 JOIN information_schema.key_column_usage AS kcu
160     ON tc.constraint_name = kcu.constraint_name
161     AND tc.table_schema = kcu.table_schema
162 JOIN information_schema.constraint_column_usage AS ccu
163     ON ccu.constraint_name = tc.constraint_name
164     AND ccu.table_schema = tc.table_schema
165 WHERE tc.constraint_type = 'FOREIGN KEY'
166 AND tc.table_schema = 'public'
167 ORDER BY tc.table_name;

```

Data Output:

	table_name name	constraint_name name	column_name name	referenced_table name	referenced_column name
1	baggage	fk_baggage_booking	booking_id	booking	booking_id
2	baggage_che...	fk_baggage_check_booking	booking_id	booking	booking_id
3	baggage_che...	fk_baggage_check_passenger	passenger_id	passengers	passenger_id
4	boardina_pass	fk_boardina_pass_booking	bookina_id	bookina	bookina_id

Total rows: 17 Query complete 00:00:00.151 CRLF Ln 167, Col 24

PART III. DML - SAMPLE DATA

I used a small sample dataset with 5 rows in the main tables. The idea is that the data can be inserted after the tables and foreign keys are created.

Airline

```

INSERT INTO airline (airline_id, airline_code, airline_name, airline_country,
created_at, updated_at)

```

```
VALUES
(1, 'KZ1', 'Kazakhstan Airlines', 'Kazakhstan', '2026-09-15 10:00:00', '2026-09-15 10:00:00'),
(2, 'FS2', 'FlyStep', 'Kazakhstan', '2026-09-15 10:00:00', '2026-09-15 10:00:00'),
(3, 'SN3', 'SkyNomad', 'Kazakhstan', '2026-09-15 10:00:00', '2026-09-15 10:00:00'),
(4, 'TL4', 'Turkic Line', 'Turkey', '2026-09-15 10:00:00', '2026-09-15 10:00:00'),
(5, 'AT5', 'Asia Travel Air', 'Uzbekistan', '2026-09-15 10:00:00', '2026-09-15 10:00:00');
```

Airport

```
INSERT INTO airport (airport_id, airport_name, country, state, city, created_at, updated_at)
VALUES
(101, 'Almaty International Airport', 'Kazakhstan', 'Almaty', 'Almaty', '2026-09-15 10:00:00', '2026-09-15 10:00:00'),
(102, 'Astana International Airport', 'Kazakhstan', 'Astana', 'Astana', '2026-09-15 10:00:00', '2026-09-15 10:00:00'),
(103, 'Aktau Airport', 'Kazakhstan', 'Mangystau', 'Aktau', '2026-09-15 10:00:00', '2026-09-15 10:00:00'),
(104, 'Shymkent Airport', 'Kazakhstan', 'Turkistan', 'Shymkent', '2026-09-15 10:00:00', '2026-09-15 10:00:00'),
(105, 'Tashkent International Airport', 'Uzbekistan', 'Tashkent', 'Tashkent', '2026-09-15 10:00:00', '2026-09-15 10:00:00');
```

Passengers

```
INSERT INTO passengers (passenger_id, first_name, last_name, date_of_birth, gender, country_of_citizenship, country_of_residence, passport_number, created_at, updated_at)
VALUES
(1, 'Ayan', 'Karimov', '2005-01-15', 'Male', 'Kazakhstan', 'Kazakhstan', 'P1001', '2026-09-15 10:00:00', '2026-09-15 10:00:00'),
(2, 'Dana', 'Karimova', '2006-03-20', 'Female', 'Kazakhstan', 'Kazakhstan', 'P1002', '2026-09-15 10:00:00', '2026-09-15 10:00:00'),
(3, 'Madi', 'Sarsenov', '2004-11-11', 'Male', 'Kazakhstan', 'Kazakhstan', 'P1003', '2026-09-15 10:00:00', '2026-09-15 10:00:00'),
(4, 'Aliya', 'Nur', '2005-06-18', 'Female', 'Kazakhstan', 'Kazakhstan', 'P1004', '2026-09-15 10:00:00', '2026-09-15 10:00:00'),
(5, 'Timur', 'Bek', '2003-08-09', 'Male', 'Kazakhstan', 'Kazakhstan', 'P1005', '2026-09-15 10:00:00', '2026-09-15 10:00:00');
```

Flights

```
INSERT INTO flights (flight_id, sch_departure_time, sch_arrival_time, departing_airport_id, arriving_airport_id, departing_gate, arriving_gate, airline_id, act_departure_time, act_arrival_time, created_at, updated_at)
VALUES
(1001, '2026-10-01 08:00:00', '2026-10-01 09:20:00', 101, 102, 'A12', 'B04', 1, '2026-10-01 08:05:00', '2026-10-01 09:25:00', '2026-09-15 10:00:00', '2026-09-15 10:00:00'),
(1002, '2026-10-02 10:00:00', '2026-10-02 13:00:00', 101, 103, 'A08', 'C11', 2, '2026-10-02 10:02:00', '2026-10-02 13:04:00', '2026-09-15 10:00:00', '2026-09-15 10:00:00'),
(1003, '2026-10-03 14:00:00', '2026-10-03 15:10:00', 102, 104, 'B07', 'D02', 3, '2026-10-03 14:00:00', '2026-10-03 15:12:00', '2026-09-15 10:00:00', '2026-09-15 10:00:00');
```

```
(1004, '2026-10-04 16:00:00', '2026-10-04 17:30:00', 104, 101, 'D05', 'A10', 4,
'2026-10-04 16:10:00', '2026-10-04 17:40:00', '2026-09-15 10:00:00', '2026-09-15
10:00:00'),
(1005, '2026-10-05 18:00:00', '2026-10-05 19:45:00', 101, 105, 'A03', 'E01', 5,
'2026-10-05 18:00:00', '2026-10-05 19:43:00', '2026-09-15 10:00:00', '2026-09-15
10:00:00');
```

Booking

```
INSERT INTO booking (booking_id, flight_id, passenger_id, booking_platform,
created_at, updated_at, status, ticket_price)
VALUES
(5001, 1001, 1, 'Web', '2026-09-15 11:00:00', '2026-09-15 11:00:00', 'CONFIRMED',
45000.00),
(5002, 1001, 2, 'Web', '2026-09-15 11:05:00', '2026-09-15 11:05:00', 'CONFIRMED',
45000.00),
(5003, 1002, 3, 'Mobile', '2026-09-15 11:10:00', '2026-09-15 11:10:00',
'CONFIRMED', 38000.00),
(5004, 1002, 4, 'Mobile', '2026-09-15 11:12:00', '2026-09-15 11:12:00',
'CONFIRMED', 38000.00),
(5005, 1003, 5, 'Agency', '2026-09-15 11:15:00', '2026-09-15 11:15:00',
'CONFIRMED', 32000.00);
```

Booking_flight

```
INSERT INTO booking_flight (booking_flight_id, booking_id, flight_id, created_at,
updated_at)
VALUES
(9001, 5001, 1001, '2026-09-15 11:00:00', '2026-09-15 11:00:00'),
(9002, 5002, 1001, '2026-09-15 11:05:00', '2026-09-15 11:05:00'),
(9003, 5003, 1002, '2026-09-15 11:10:00', '2026-09-15 11:10:00'),
(9004, 5004, 1002, '2026-09-15 11:12:00', '2026-09-15 11:12:00'),
(9005, 5005, 1003, '2026-09-15 11:15:00', '2026-09-15 11:15:00');
```

Baggage_check

```
INSERT INTO baggage_check (baggage_check_id, check_result, created_at, updated_at,
booking_id, passenger_id)
VALUES
(7001, 'PASSED', '2026-09-15 12:00:00', '2026-09-15 12:00:00', 5001, 1),
(7002, 'PASSED', '2026-09-15 12:01:00', '2026-09-15 12:01:00', 5002, 2),
(7003, 'PASSED', '2026-09-15 12:02:00', '2026-09-15 12:02:00', 5003, 3),
(7004, 'PASSED', '2026-09-15 12:03:00', '2026-09-15 12:03:00', 5004, 4),
(7005, 'PASSED', '2026-09-15 12:04:00', '2026-09-15 12:04:00', 5005, 5);
```

Baggage

```
INSERT INTO baggage (baggage_id, weight_in_kg, created_at, updated_at, booking_id)
VALUES
(8001, 12.50, '2026-09-15 12:10:00', '2026-09-15 12:10:00', 5001),
(8002, 15.00, '2026-09-15 12:11:00', '2026-09-15 12:11:00', 5002),
(8003, 9.80, '2026-09-15 12:12:00', '2026-09-15 12:12:00', 5003),
(8004, 18.20, '2026-09-15 12:13:00', '2026-09-15 12:13:00', 5004),
(8005, 7.40, '2026-09-15 12:14:00', '2026-09-15 12:14:00', 5005);
```

Boarding_pass

```
INSERT INTO boarding_pass (boarding_pass_id, booking_id, seat, boarding_time,
created_at, updated_at)
```

```
VALUES
(6001, 5001, '12A', '2026-10-01 07:20:00', '2026-09-15 12:20:00', '2026-09-15
12:20:00'),
(6002, 5002, '12B', '2026-10-01 07:20:00', '2026-09-15 12:21:00', '2026-09-15
12:21:00'),
(6003, 5003, '8C', '2026-10-02 09:20:00', '2026-09-15 12:22:00', '2026-09-15
12:22:00'),
(6004, 5004, '8D', '2026-10-02 09:20:00', '2026-09-15 12:23:00', '2026-09-15
12:23:00'),
(6005, 5005, '4A', '2026-10-03 13:20:00', '2026-09-15 12:24:00', '2026-09-15
12:24:00');
```

Security_check

```
INSERT INTO security_check (security_check_id, check_result, created_at,
updated_at, passenger_id)
VALUES
(3001, 'PASSED', '2026-09-15 12:30:00', '2026-09-15 12:30:00', 1),
(3002, 'PASSED', '2026-09-15 12:31:00', '2026-09-15 12:31:00', 2),
(3003, 'PASSED', '2026-09-15 12:32:00', '2026-09-15 12:32:00', 3),
(3004, 'PASSED', '2026-09-15 12:33:00', '2026-09-15 12:33:00', 4),
(3005, 'PASSED', '2026-09-15 12:34:00', '2026-09-15 12:34:00', 5);
```

The screenshot shows a database management interface with a query editor and a results pane. The query editor contains SQL commands for inserting data into the `Boarding_pass` and `Security_check` tables, followed by a comment and a `SELECT` statement to view the results of the insert.

Query History:

```

204 (8003, 9.80, '2026-09-15 12:12:00', '2026-09-15 12:12:00', 5003),
205 (8004, 18.20, '2026-09-15 12:13:00', '2026-09-15 12:13:00', 5004),
206 (8005, 7.40, '2026-09-15 12:14:00', '2026-09-15 12:14:00', 5005);
207
208 INSERT INTO Boarding_pass (boarding_pass_id, booking_id, seat, boarding_time, created_at, upda
209 (6001, 5001, '12A', '2026-10-01 07:20:00', '2026-09-15 12:20:00', '2026-09-15 12:20:00'),
210 (6002, 5002, '12B', '2026-10-01 07:20:00', '2026-09-15 12:21:00', '2026-09-15 12:21:00'),
211 (6003, 5003, '8C', '2026-10-02 09:20:00', '2026-09-15 12:22:00', '2026-09-15 12:22:00'),
212 (6004, 5004, '8D', '2026-10-02 09:20:00', '2026-09-15 12:23:00', '2026-09-15 12:23:00'),
213 (6005, 5005, '4A', '2026-10-03 13:20:00', '2026-09-15 12:24:00', '2026-09-15 12:24:00');
214
215 INSERT INTO Security_check (security_check_id, check_result, created_at, updated_at, passenger
216 (3001, 'PASSED', '2026-09-15 12:30:00', '2026-09-15 12:30:00', 1),
217 (3002, 'PASSED', '2026-09-15 12:31:00', '2026-09-15 12:31:00', 2),
218 (3003, 'PASSED', '2026-09-15 12:32:00', '2026-09-15 12:32:00', 3),
219 (3004, 'PASSED', '2026-09-15 12:33:00', '2026-09-15 12:33:00', 4),
220 (3005, 'PASSED', '2026-09-15 12:34:00', '2026-09-15 12:34:00', 5);
221
222 -- 6. Вывод результата вставки
223 SELECT * FROM Booking;
```

Data Output:

	booking_id [PK] integer	flight_id integer	passenger_id integer	booking_platform character varying (50)	created_at timestamp without time zone	updated_at timestamp without time zone	status character varying (50)	ticket_price numeric (7,2)
1	5001	1001	1	Web	2026-09-15 11:00:00	2026-09-15 11:00:00	CONFIRMED	45000.00
2	5002	1001	2	Web	2026-09-15 11:05:00	2026-09-15 11:05:00	CONFIRMED	45000.00
3	5003	1002	3	Mobile	2026-09-15 11:10:00	2026-09-15 11:10:00	CONFIRMED	38000.00
4	5004	1002	4	Mobile	2026-09-15 11:12:00	2026-09-15 11:12:00	CONFIRMED	38000.00

Total rows: 5 Query complete 00:00:00.234 CRLF Ln 223, Col 23

Conclusion

In this lab I learned how 1NF, 2NF and 3NF are different. First I made all values atomic, then I removed partial dependencies and finally I separated data with transitive dependencies. After that I used DDL commands to create the airport database, change the tables and add foreign keys. The final database structure has less duplicated data and is easier to maintain.